

Environment Configuration

Music Podcast Player

1 INTRODUCTION

This document describes the environment configuration required to run Music Podcast Player. It focuses on configuration files, environment variables, local service connections, and deployment-specific settings. Full instructions for deploying the application locally or to AWS are covered separately in the Deployment Instructions document.

2 FRONTEND CONFIGURATION

The frontend uses Vue 3 with Vite. Some frontend environment variables are written into the frontend build when the application is built. Because of this, these values cannot be changed on an already active deployment without rebuilding the frontend.

2.1 FRONTEND BUILD-TIME VARIABLES

- **VITE_INCLUDE_DEV_FEATURES** – Set to either **yes** or **no**. When set to **yes**, the frontend enables pages that are meant for testing development features and are not intended for ordinary users.
- **VITE_BLOCK_LIGHTNING_PAYMENTS** – Set to either **yes** or **no**. When set to **yes**, Lightning payments are not actually sent. Instead, payment details are printed to the console. This is useful for testing payment-related features without connecting to the Lightning Network.
- **VITE_LIGHTNING_RECIPIENT_OVERRIDE** – Either unset or set to a Lightning node address. When configured, this allows payment features to be tested with real Lightning wallets on the regtest network.

2.2 FRONTEND RUNTIME CONFIGURATION

Runtime frontend configuration is handled through `config.json`. This file is used for values that must be configurable after the frontend has already been built.

- **backendUrl** – The URL the frontend uses to connect to the API service. In a local deployment, this is usually set to `http://localhost:5000`. In an AWS deployment, this value is generated by the CDK and stored alongside the frontend in S3.

3 API SERVICE CONFIGURATION

The API service handles frontend requests and connects to the internal database service, Podcast Index API, DynamoDB, and S3-compatible storage. Its configuration is provided through environment variables.

- `RSS_PLAYER_ALLOWED_ORIGINS` – A comma-separated list of URLs allowed for Cross-Origin Resource Sharing (CORS). In local development, this includes the frontend URL.
- `RSS_PLAYER_ENVIRONMENT` – Set to either `production` or `development`. This is currently used to determine whether cookies should be marked as secure and only sent over HTTPS.
- `RSS_PLAYER_DB_SERVICE_URL` – The URL the API service uses to reach the internal database service.
- `RSS_PLAYER_API_ROOT` – The prefix for the API service’s endpoints, such as `/api/v1`.

4 API SECRETS AND EXTERNAL SERVICE CREDENTIALS

Some API service values are sensitive and should not be stored directly in the repository. In local development, these are stored in `secrets/backend.env`. In AWS deployments, several secrets are fetched from AWS Systems Manager Parameter Store.

4.1 AWS SECRET ROUTE VARIABLES

- `PODCAST_INDEX_KEY_ROUTE` – Only used in AWS deployments. This identifies where to fetch the Podcast Index API key from System Manager.
- `PODCAST_INDEX_SECRET_ROUTE` – Only used in AWS deployments. This identifies where to fetch the Podcast Index API secret from Systems Manager.
- `SECRET_KEY_ROUTE` – Only used in AWS deployments. This identifies where to fetch the secret key used for JSON Web Tokens (JWTs).

4.2 LOCAL SECRET VARIABLES

- `RSS_PLAYER_PODCAST_INDEX_KEY` – The Podcast Index API key. This is only manually set for local development and should be stored in `secrets/backend.env`.
- `RSS_PLAYER_PODCAST_INDEX_SECRET` – The Podcast Index API secret. This is only manually set for local development and should be stored in `secrets/backend.env`.
- `RSS_PLAYER_SECRET_KEY` – The secret key used to sign JWTs. This is only manually set for local development.

The Podcast Index credentials are live credentials, even in local development, and should never be committed to the repository.

5 RATE LIMITING CONFIGURATION

Music Podcast Player uses token-based rate limiting to help control API usage and limit requests to external services such as the Podcast Index API.

- `RSS_PLAYER_DYNAMODB_URL` – Only set for local development. This is the URL the API service uses to reach the local DynamoDB container.

- `RSS_PLAYER_API_TOKENS_PER_REFILL` – A JSON object defining how many rate-limit tokens each user group receives per refill period. The `global` group applies across the whole app, the `public` group applies to unauthenticated users, and the `user` group applies to each authenticated user individually. The `overall` token category is consumed by general API requests, while the `podcast_index` category is consumed by requests that require the Podcast Index API.
- `RSS_PLAYER_TOKEN_REFILL_SECONDS` – The amount of time between token refills after a user group first spends a token.
- `RSS_PLAYER_TOKEN_PENALTY` – The number of categorized tokens deducted when a request to an external service results in a `429 Too Many Requests` response.
- `RSS_PLAYER_TOKEN_TABLE_NAME` – Only set manually in local development. This is the DynamoDB table used for rate-limit token storage.

6 EMAIL AND STORAGE CONFIGURATION

Music Podcast Player includes configuration for email verification and editable page storage. Local development uses MinIO as an S3-compatible storage service.

- `RSS_PLAYER_SES_FROM_EMAIL` – The no-reply email address used when verification emails are sent to users. This is currently only supported for AWS deployments.
- `RSS_PLAYER_DISABLE_EMAIL_VERIFY` – When set to `true`, verification emails are disabled and all verification codes are treated as `000000`.
- `RSS_PLAYER_CMS_BUCKET` – Only set manually in local development. This is the S3 bucket used to store editable pages. Locally, this is handled by MinIO.
- `RSS_PLAYER_S3_ENDPOINT_URL` – Only set manually in local development. This is the URL used to reach the local S3-compatible MinIO service.
- `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_DEFAULT_REGION` – Only manually set for local development. These are used by AWS SDK tools such as boto3. In local development, they allow the application to connect to MinIO and local AWS-compatible services.

7 DATABASE SERVICE CONFIGURATION

The database service connects the backend to PostgreSQL. Local deployments use a direct database connection URL, while AWS deployments use structured database connection information and IAM authentication.

- `DATABASE_URL` – Only set manually in local development. This is the database connection URL used to connect to PostgreSQL. A typical local value is `postgresql://rssmusicplayerapp:dev-db-service-password@database:5432/postgres`.
- `DATABASE_INFO` – Only set for AWS deployments. This is a JSON object containing the database driver, address, port, PostgreSQL role, AWS region, and database name used to connect to the database. AWS deployments use IAM authentication.

8 MIGRATION CONFIGURATION

Database migrations prepare the database schema before the backend services begin handling requests.

- `DATABASE_URL` – Only set manually in local development. This is the database connection URL used by the migration service.
- `DB_SERVICE_ROLE_PASSWORD` – Only set in local development. This is the password used for the database service role when that role is created through migrations.
- `DATABASE_URL_PARTIAL` – Only set for AWS deployments. This is a partial database connection URL containing placeholders for the master role and password used when the migration function connects to the database.
- `DATABASE_CREDENTIAL` – Only set for AWS deployments. This contains the master role and password used to connect to the database. The password is generated by the CDK and rotated after a successful deployment.

9 GITHUB ACTIONS ENVIRONMENT VARIABLES

The continuous deployment workflows use GitHub environment variables to authenticate with AWS and determine where the application should be deployed.

- `AWS_DEPLOY_ROLE` – The ARN of the AWS role to assume when deploying to this environment from GitHub.
- `AWS_FETCH_ROLE` – The ARN of the AWS role to assume when running `cdk diff` without deploying. This is only set in development.
- `AWS_POST_DEPLOY_ROLE` – The ARN of the AWS role to assume when running post-deployment scripts, such as rotating the database credential.
- `AWS_REGION` – The AWS region used for deployment.
- `MPP_DOMAIN_NAME` – The domain name used for the deployment. This is only set in production. For example, `musicpodcastplayer.com`.
- `MPP_RECORD_NAME` – The subdomain used for the deployment. This is only set in production. For example, `www`.
- `MPP_R53_ZONE_ID` – The AWS Route 53 hosted zone ID for the production domain.

10 LOCAL SERVICE SUMMARY

Although most configuration is handled through environment variables, the local development environment also exposes several services on local ports.

Service	Purpose	Local Port
Frontend	Vue 3 and Vite frontend	5173
Backend API	Main backend API service	5000
PostgreSQL	Local relational database	5432
MinIO API	Local object storage API	9000
MinIO Console	Browser-based MinIO console	9001

Table 1: Local Service Summary